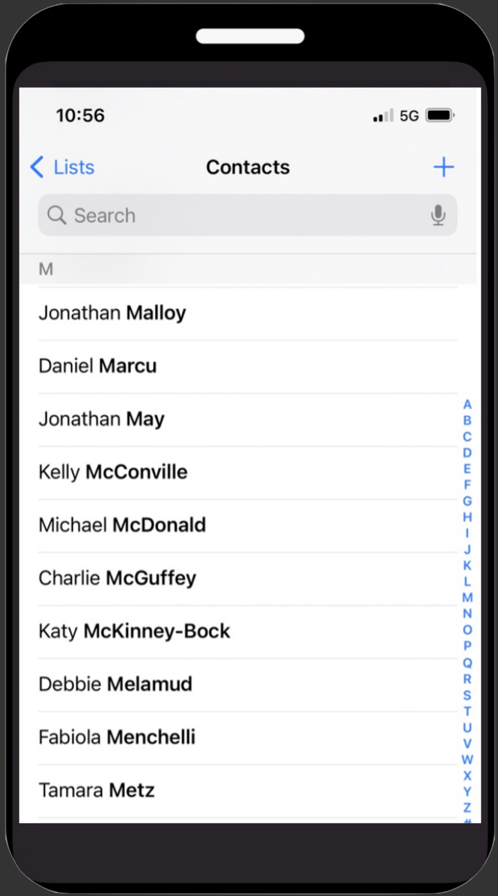
dictionaries

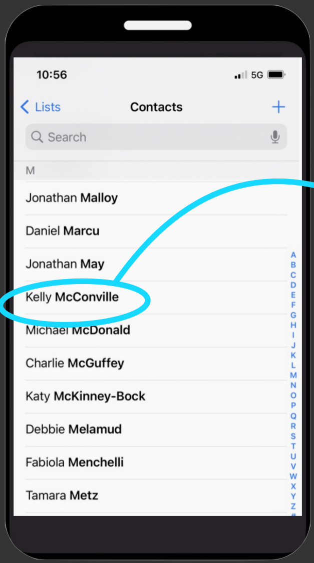
CSCI 134

a phone


10:56
5G
Lists
Contacts
Search
M
Jonathan Malloy
Daniel Marcu
Jonathan May
Kelly McConville
Michael McDonald
Charlie McGuffey
Katy McKinney-Bock
Debbie Melamud
Fabiola Menchelli
Tamara Metz
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

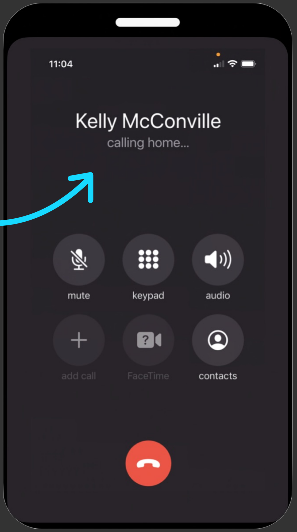
it can be used
to call people

when you tap here



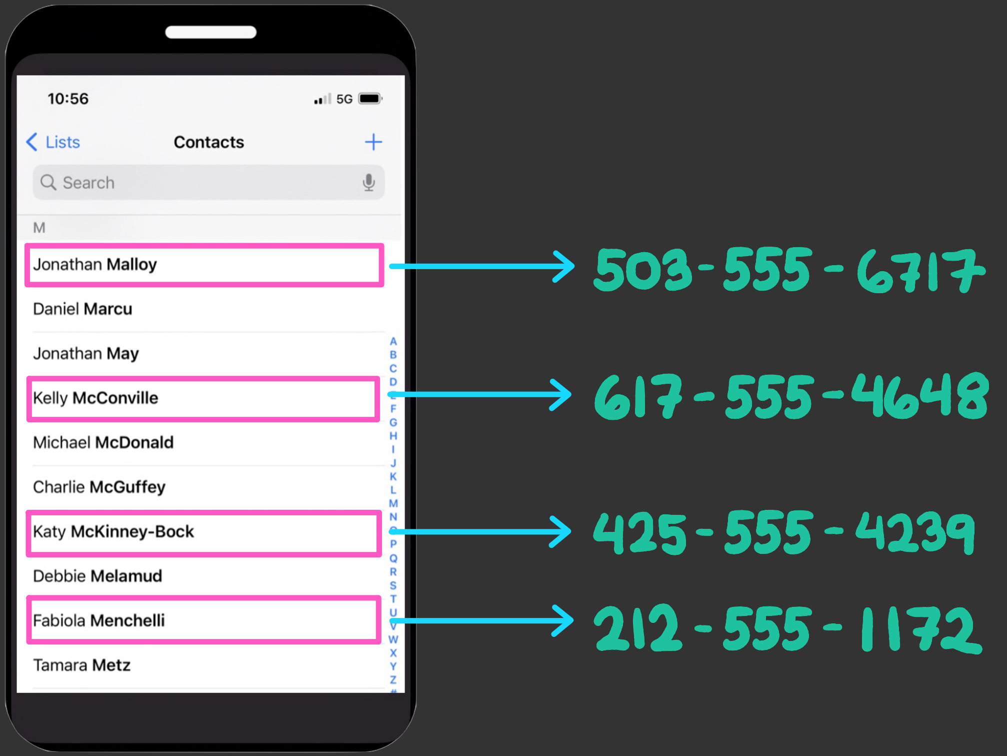
617-555-4648

the phone looks up the number associated with kelly

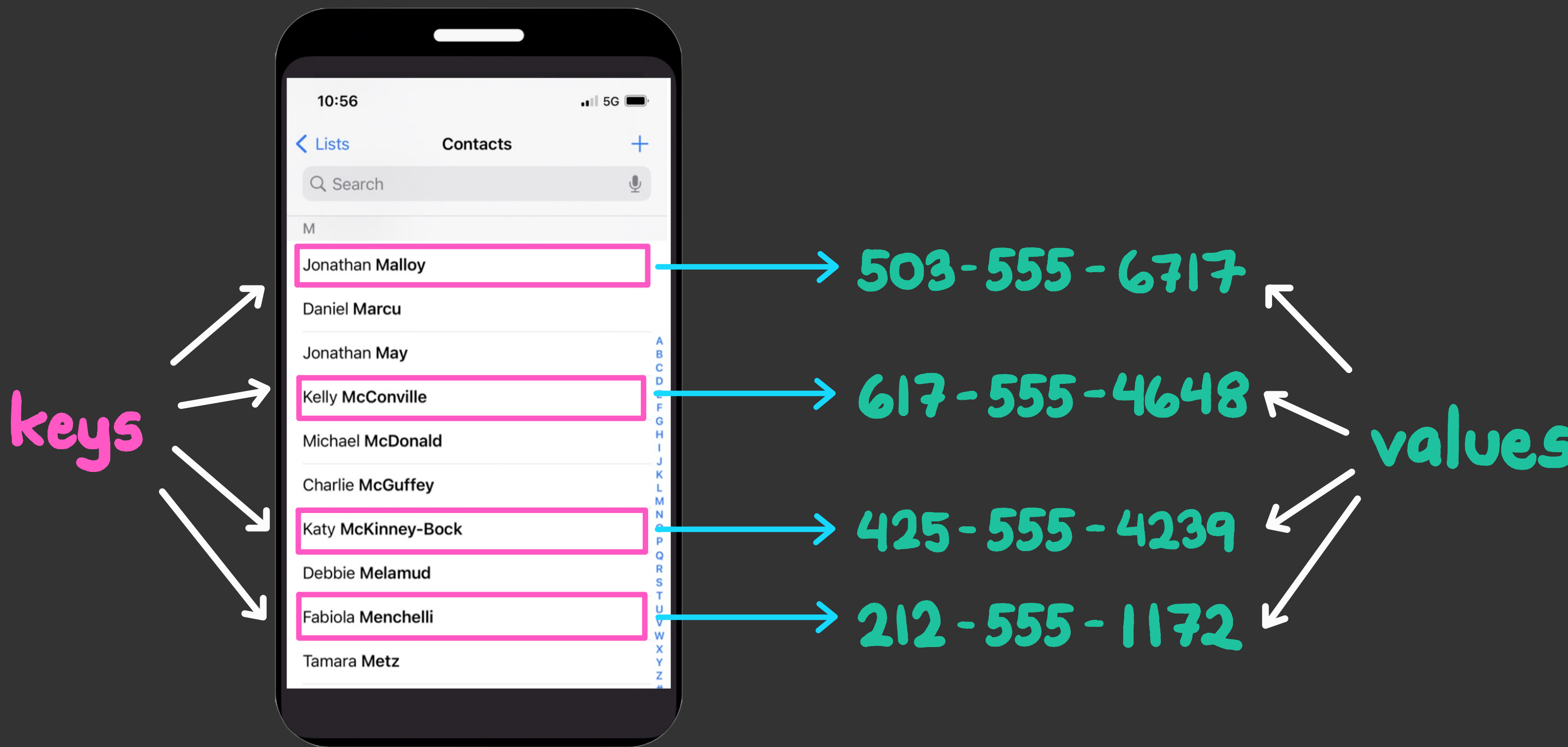


and calls her

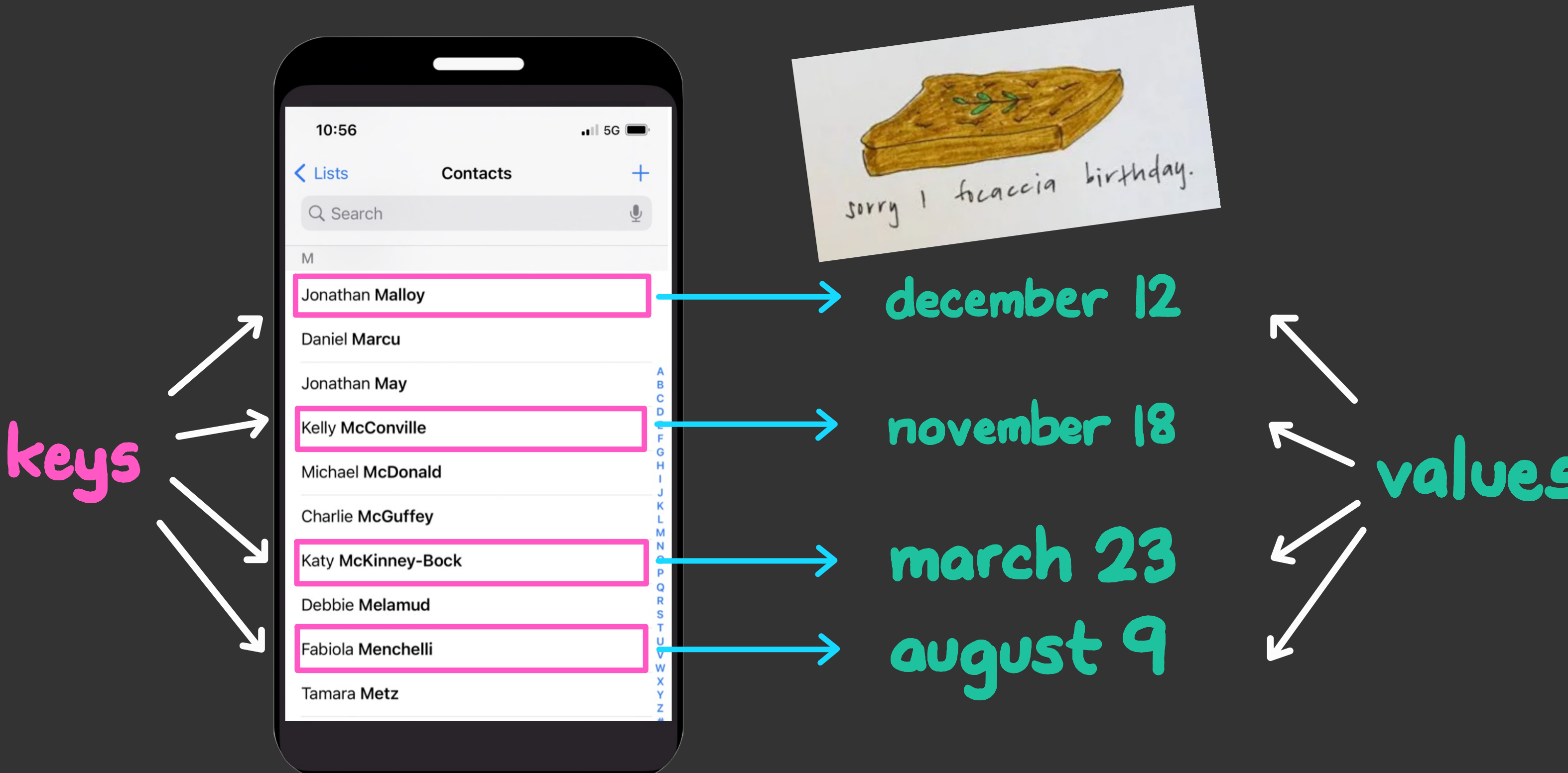
each contact is associated with a phone number



such lookup tables are quite useful



such lookup tables are quite useful

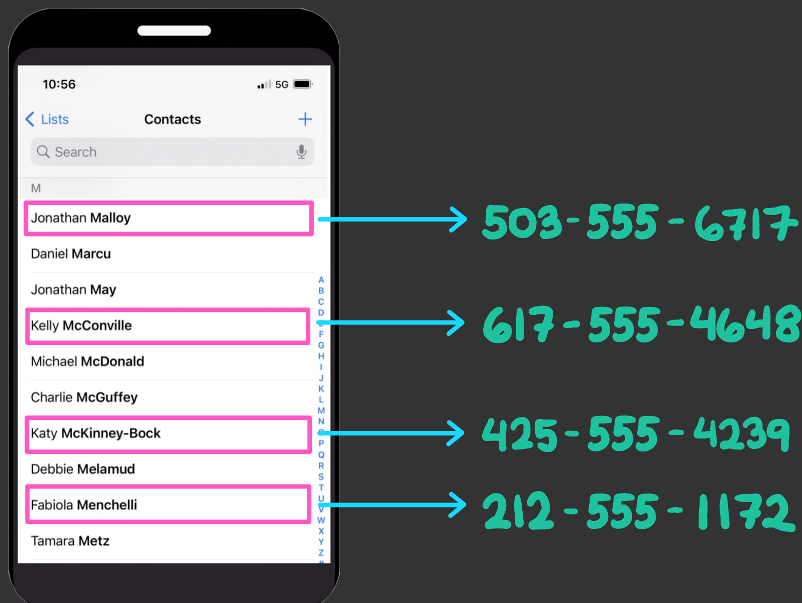


in python, lookup tables are called dictionaries

```
{'Jonathan': '503-555-6717', 'Kelly': '617-555-4648', 'Katy': '425-555-4239', 'Fabiola': '212-555-1172'}
```

in python, lookup tables are called dictionaries

{ key : value , key : value , key : value , key : value }



you can specify as many key-value pairs as you like

{ key : value , key : value , key : value , key : value }

this is an empty **dictionary**

```
In [1]: {}
Out[1]: {}
```

also an empty **dictionary**

```
In [2]: dict()
Out[2]: {}
```

the type of a dictionary is dict

```
In [1]: phone = {'Kelly': '617-555-4648', 'Fabiola': '212-555-1172'}

In [2]: type(phone)
Out[2]: dict
```

you can look up the value **associated** with a key

dict [key]

```
In [1]: phone = {'Kelly': '617-555-4648', 'Fabiola': '212-555-1172'}

In [2]: phone['Kelly']
Out[2]: '617-555-4648'

In [3]: phone['Fabiola']
Out[3]: '212-555-1172'
```

but if you ask for the value associated with a non-existent key, python will get upset

```
In [1]: phone = {'Kelly': '617-555-4648', 'Fabiola': '212-555-1172'}

In [2]: phone['Jonathan']
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[2], line 1
----> 1 phone['Jonathan']

KeyError: 'Jonathan'
```

fortunately, you can check whether a key is **in** a dictionary

```
In [1]: phone = {'Kelly': '617-555-4648', 'Fabiola': '212-555-1172'}

In [2]: 'Kelly' in phone
Out[2]: True

In [3]: 'Jonathan' in phone
Out[3]: False
```

this only works for keys, not values

```
In [1]: phone = {'Kelly': '617-555-4648', 'Fabiola': '212-555-1172'}

In [2]: 'Kelly' in phone
Out[2]: True

In [3]: '617-555-4648' in phone
Out[3]: False
```

you can change the value associated with a key

```
dict [ key ] = expression
```

```
In [1]: phone = {'Kelly': '617-555-4648', 'Fabiola': '212-555-1172'}

In [2]: phone['Kelly']
Out[2]: '617-555-4648'

In [3]: phone['Fabiola']
Out[3]: '212-555-1172'

In [4]: phone['Kelly'] = '503-555-1234'

In [5]: phone['Kelly']
Out[5]: '503-555-1234'

In [6]: phone
Out[6]: {'Kelly': '503-555-1234', 'Fabiola': '212-555-1172'}
```

this also allows you to add a **new** key-value pair

```
dict[key] = expression
```

```
In [1]: phone = {'Kelly': '617-555-4648', 'Fabiola': '212-555-1172'}

In [2]: phone['Jonathan'] = '503-555-6717'

In [3]: phone
Out[3]:
{'Kelly': '617-555-4648',
 'Fabiola': '212-555-1172',
 'Jonathan': '503-555-6717'}

In [4]: phone['Jonathan']
Out[4]: '503-555-6717'

In [5]: len(phone)
Out[5]: 3
```

the number of key-value pairs in the dict you can **iterate** through the keys of a dict using a **for** statement

```
for variable in dict:
    python code
```

```
In [1]: phone = {'Kelly': '617-555-4648', 'Fabiola': '212-555-1172'}

In [2]: for key in phone:
   ...:     print(key + " --> " + phone[key])
   ...:
Kelly --> 617-555-4648
Fabiola --> 212-555-1172
```

two dicts are equal if they have the same key-value pairs

```
In [1]: phone = {'Kelly': '617-555-4648', 'Fabiola': '212-555-1172'}

In [2]: phone2 = {'Fabiola': '212-555-1172', 'Kelly': '617-555-4648'}

In [3]: phone == phone2
Out[3]: True

In [4]: phone3 = {'Fabiola': '123-456-7890', 'Kelly': '617-555-4648'}

In [5]: phone == phone3
Out[5]: False
```

order doesn't matter

expressions so far

type	example
int	7
float	3.14
str	"williams"
bool	3 > 4
function	abs
NoneType	print("hi")
range	range(1,5)
list	["mh", "rb", "jb"]
dict	{"a": 1, "b": 2, "c": 3}